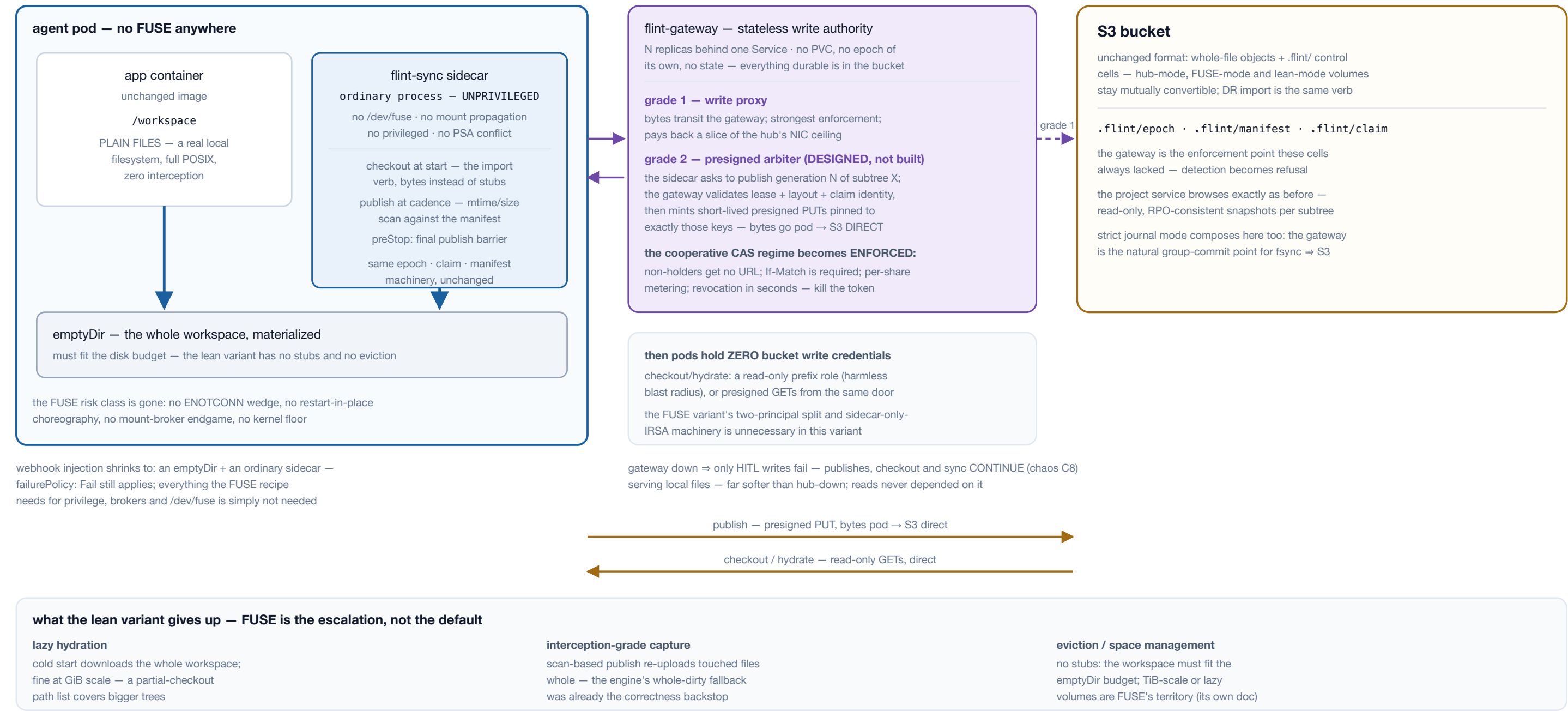


The third front end over the flint bucket format, split out of the flint-fuse architecture doc: if the workspace fits on disk, no interception is needed at all. Materialize plain files at pod start, publish changed files at cadence with an unprivileged sidecar, and put write authority in a stateless gateway. Most of FUSE's value at a fraction of its risk surface — same bucket format, same engine.



This is a third front end, not a FUSE configuration — the app touches plain files on a real local filesystem, and the sidecar is an ordinary unprivileged program running the same engine verbs: **import** at start (bytes instead of stubs), the flush barrier's whole-file lane at cadence (driven by an mtime/size scan against the manifest instead of capture interception), the same epoch, claim, and manifest machinery. For the agent-workspace shape — code, configs, artifacts, GiB-scale — it is most of FUSE's value with none of FUSE's hard parts: the entire privileged/mount-propagation/ENOTCONN/mount-broker risk section of the FUSE architecture stops applying, and the kernel floor drops to nothing. **Two corrections from the drills.** This page originally read “gateway down ⇒ publishes pause”; chaos leg C8 measured the opposite — barriers continue, because the shipped sidecar writes its manifest, window and epoch cells straight to the store. All four stated outage effects belong to the **proxy**; the gateway costs only the fourth. And **grade 2 is a design, not a build** — nothing in the tree mints a presigned URL today, and the shipped path gives the sidecar a scoped credential of its own (the agent container still holds none). **The gateway restores the security posture the review said direct mode gives up.** With grade 2, “the bucket trusts one principal” is true again — pods hold no write credentials, the cooperative CAS regime becomes enforced arbitration (a non-holder is refused a URL, not merely detected after the fact), revocation returns to seconds, and per-share metering exists again. Its failure mode is publish-pause with growing RPO while pods keep serving — the softest failure shape any component on these pages has. **One dialect everywhere:** this also argues for converging the hub's file API onto the S3 REST surface (the libflint plan's §8 sidecar adapter, built once, placed three ways — hub listener, localhost sidecar, gateway), so the live view and the published view speak the same protocol and the same tools work against both. **Recommendation:** lean is the direct-mode entry point; FUSE is the escalation for volumes too large or too lazy to materialize; the hub is unchanged. Same bucket, three front ends, one engine. Implementation plan: docs/plans/flint-lean-plan.md.

The decision of record: every REST surface in the system speaks the S3 dialect — the hub's file API converges onto it, the gateway serves it, and direct-mode non-mount access (lean and FUSE alike) is plain S3 against the bucket. One library, three placements; the consistency contracts say which view a client is reading.



Pages 1–2 describe publishing **at cadence**: an mtime/size scan on a timer. That is still the default, and it is still the whole story for a workspace nobody is watching. What it could not express is a coherent point — “these files, together, now” — so an agent finishing a unit of work had no way to say so and no way to learn when its bytes were durable. The boundary verbs are that sentence, written as files in the workspace the agent already has. Nothing new is mounted, no port is opened, and a workspace that ignores them pays nothing: measured at **20 bucket requests per 22 s idle with the verbs off and 20 with them on**.

